Plan de travail à 4

Projet Signaux · suivi de consommation par lecture de compteur · Rôles, plan A (sans Pico, 10 jours) et plan B (avec Pico)
Version du 21 octobre 2026 · même contenu que le wiki du dépôt

Organisation à 4

Chacun est **responsable** d'une partie, mais tout le monde aide les autres. Le responsable, c'est celui qui s'assure que sa partie est finie à temps.

Les 4 rôles

Rôle	Responsable de	Fichiers	Pages du wiki à lire
A — Données	Prendre les photos, noter le vrai index de chaque photo, les mettre dans la base	`static/images/`, fichier CSV d'annotation	*Prendre et annoter les photos*, *Base de données*
B — Traitement d'image	Préparer les photos : recadrer, gris, seuillage, découper les chiffres	`classes/` (classe de lecture)	*Traitement d'image*
C — Reconnaissance + validation	Transformer chaque chiffre découpé en nombre, mesurer le taux de réussite	`classes/` (classe de lecture), tableau de résultats	*Traitement d'image*, *Validation et robustesse*
D — Site et base	Supabase, calculs (moyenne, min, pic), routes Flask, page avec le graphique	`app.py`, `templates/`, `static/js/`, `static/css/`	*Le site Flask*, *Base de données*

À la fin, **tout le monde** travaille sur le poster et la démo.

Qui dépend de qui

- **A** doit fournir des photos annotées **très tôt** : sans elles, B et C ne peuvent rien tester.
- **B** donne à **C** des images de chiffres bien nettes (blanc sur noir).
- **C** donne à **D** un index lu pour chaque photo.
- **D** peut avancer **seul dès le début** : il travaille avec le vrai index noté à la main en attendant que la lecture automatique marche.

Règles d'équipe

☐ Avant de travailler : **Pull** (récupérer le travail des autres).
☐ Une branche Git par tâche, fusion dans `main` quand ça marche.
☐ Chacun travaille surtout dans **ses** fichiers : moins de conflits.
☐ Ne **jamais** mettre de mot de passe ou de clé dans Git (fichier `.env` + `.gitignore`).
☐ Un point de 10 minutes chaque matin : ce que j'ai fini, ce que je fais, ce qui me bloque.
☐ Messages de commit clairs : « lecture : découpage des chiffres », « site : ajoute le minimum ».

Livrables et responsable

☐ Slides de l'état des lieux (22/10) — tout le monde, une slide chacun
☐ Photos annotées (≥ 150) — A
☐ Lecture de l'index — B + C
☐ Site avec moyenne, minimum, pic — D
☐ Tableau de validation — C
☐ Poster A1 — tout le monde, D assemble
☐ Code ZIP + README — D
☐ Vidéo de démo de secours — A

Plan A — sans Pico (le plan qu'on suit)

Budget : 2 semaines à temps plein = 10 jours de travail. On a 4 projets en parallèle, donc on vise le minimum demandé, bien fait et bien validé. Rien de plus.

Les 10 jours sont à placer avant le 30 novembre, en gardant **au moins 2 jours de marge**. Par exemple : du lundi 9 au vendredi 20 novembre.

Avant les 10 jours (dès cette semaine, quelques heures)

☐ Présenter l'état des lieux le jeudi 22/10 à 13h30.
☐ Créer le projet Supabase et la table (D) — voir *Base de données*.
☐ Commencer les photos (A). Le compteur avance lentement : il faut étaler les séances sur plusieurs jours de toute façon.
☐ Chacun installe le projet chez lui et lance le site vide.

Semaine 1 — de la photo au nombre

Jour	A — Données	B — Traitement	C — Reconnaissance	D — Site et base
J1	Finir l'annotation (CSV), mettre 1 photo sur 3 de côté pour le test	Ouvrir les photos, trouver la zone des chiffres	Lire la doc ssocr, tester à la main sur 5 photos	Table Supabase prête, connexion depuis Python
J2	Importer toutes les photos dans la base	Gris + flou + seuillage : chiffres blancs nets	Préparer la table des 7 segments	Calcul conso + moyenne, min, pic (avec le vrai index)
J3	Aider B : photos difficiles (reflet, angle)	Découper chaque chiffre	Reconnaître un chiffre seul	Route `/api/data` qui renvoie du JSON
J4	Aider C : vérifier les résultats à la main	Gérer les chiffres penchés / collés	Reconnaître un nombre complet (avec la virgule)	Page avec Chart.js qui affiche les vraies données
J5	**Tous ensemble** : brancher la lecture → base → site. À la fin du jour, le graphique s'affiche avec l'index lu par le programme.			

Semaine 2 — vérifier et rendre

Jour	A — Données	B — Traitement	C — Reconnaissance	D — Site et base
J6	Préparer la démo (série de photos qui montre bien un pic)	Corriger les erreurs de prétraitement	Mesurer sur les photos de test, faire le tableau par condition	Garde-fou (index qui baisse) + affichage des rejets
J7	Analyse des erreurs : **tous ensemble**, une seule amélioration ciblée			
J8	Enregistrer la vidéo de démo de secours	Nettoyer et commenter son code	Graphiques de résultats pour le poster	Finitions du site, README

Jour	A — Données	B — Traitement	C — Reconnaissance	D — Site et base
J9	**Poster A1** : tout le monde écrit sa partie, D assemble			
J10	Relecture, ZIP du code (sans mot de passe), dépôt Moodle			

Ce qu'on ne fait pas dans le plan A

- Le Pico et la caméra.
- La détection automatique de la zone des chiffres.
- Les compteurs à rouleaux.

Si tout est fini en avance → voir le plan B.

Si on prend du retard

On coupe dans cet ordre :

1. L'amélioration du J7.
2. Les finitions du site.
3. Si la lecture automatique ne marche pas assez bien : on la présente **telle quelle** avec son taux de réussite et l'analyse des erreurs. Un résultat honnête et bien analysé vaut mieux qu'une démo truquée.

Plan B — avec Pico (seulement si le plan A est fini)

On ne commence pas tant que le plan A n'est pas terminé. Compter environ 4 à 5 jours de plus.

L'idée

Le Pico W remplace le téléphone. Toutes les 5 minutes, il prend une photo avec l'Arducam et l'envoie au site Flask par Wi-Fi. Le site l'enregistre et la lit, comme une photo du téléphone.

- La **base ne change pas** : chaque photo devient une ligne de plus.
- La **lecture des chiffres reste sur le PC** : le Pico a trop peu de mémoire pour faire du traitement d'image.
- La route `/upload` existe déjà dans `app.py` : c'est elle qui recevra les photos.

Les étapes

Étape	Qui	Objectif	C'est fini quand…
B1	A + B	Brancher l'Arducam au Pico W, prendre **une** photo	On a un fichier JPEG lisible
B2	A	Connecter le Pico au Wi-Fi	Le Pico affiche son adresse IP
B3	A + D	Envoyer la photo au PC en HTTP POST	Flask reçoit le fichier sur `/upload`
B4	D	Dans `/upload` : sauvegarder la photo, ajouter la ligne, lancer la lecture	La photo apparaît sur le site
B5	A	Répéter toutes les 5 min, fixer le Pico devant le compteur	Une nuit de photos sans intervention
B6	C	Comparer les résultats Pico et téléphone	Une ligne de plus dans le tableau de validation

Où chercher

- **Arducam** : la documentation de **votre modèle exact** (Arducam Mega SPI ou autre). Arducam fournit des exemples pour le Pico. Vérifier qu'ils existent en MicroPython avant de choisir.
- **Wi-Fi en MicroPython** : module `network`, classe `WLAN`, méthodes `active()`, `connect()`, `isconnected()`.
- **Envoi HTTP** : module `urequests` (ou `requests` selon la version), fonction `post()`.
- **Attendre 5 minutes** : `time.sleep()`, ou mieux une boucle avec `machine.deepsleep()` pour économiser.
- **Côté Flask** : `request.files` pour récupérer le fichier envoyé, méthode `save()` du fichier.

Pièges

- Flask doit écouter sur le réseau, pas seulement sur le PC : chercher le paramètre `host` de `app.run()`.
- Le PC et le Pico doivent être sur le **même Wi-Fi**.

- Protéger `/upload` avec un petit mot de passe partagé, sinon n'importe qui sur le réseau peut envoyer des images.
- L'image de l'Arducam est moins bonne que celle d'un téléphone : il faudra re-régler la zone des chiffres et le seuillage.
- Noter quand même le **vrai index** sur un échantillon de photos du Pico, sinon on ne peut pas valider.
- Le code du Pico va dans `MicroPython/main.py`.